

Artificial Intelligence Exam Notes

CT 710 (BEI, IV-I) · CT 653 (BCT, III-II) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

3. Search Techniques

9 hrs · asked in **all 41 papers** · 3 syllabus sub-topics · **392 marks, 11.6% of the archive** · worked traces in Ch3 – Search Techniques – Solved Problems.

Read this first:

- **The chapter is three comparisons and one algorithm.** Learn **BFS Vs DFS** (TOP 10/41), **informed Vs uninformed** (HOT 4/41), **A* Vs greedy** (HOT 7/41), and the **A* algorithm** itself, and you have answered most of what this chapter has ever asked.
- **One table carries the whole of §3.2.** Completeness, optimality, time, space, for BFS and DFS. It is asked in **10 of the 41 papers** and it is four rows. Nothing else in the subject pays as well per line learnt.
- The other three reliable topics: **hill climbing's three problems and their cures** (HOT 6/41), **minimax + alpha-beta** (HOT 6/41), and **why searching is necessary in AI** (HOT 5/41), which is a 2-mark opener bolted onto a bigger question five times.
- **Simulated annealing is asked four times and is not a syllabus line.** It only ever appears as “how does simulated annealing overcome the problems of hill climbing”, so learn it as hill climbing's cure, not as a topic.
- Five papers print a **figure** and ask you to run an algorithm on it: A* on a graph (81 Ch, 80 Ch), best-first on a tree (76 Bh), minimax with alpha-beta on a tree (79 Ch, 76 Ba). All five are worked in the companion. **Two of those papers print the same graph** with different heuristics.
- Pairs the examiner reliably asks together: **why search + BFS Vs DFS, informed Vs uninformed + hill climbing**, and **greedy's drawback + how A* fixes it**.

3.1 Why search, and how a search algorithm is judged

Why is searching necessary in AI? On what basis are search algorithms evaluated? 2 3 4

(HOT 5/41) 72 Ma 2, 74 Bh 2, 78 Ch 2, 80 Ash 2, 82 Ka 3 · evaluation criteria alone in 3 papers, 79 Ash 4, 75 Bh 4, 81 Ash 2, none over 4 marks so no tier chip · **always the 2-mark opener** to a BFS/DFS or hill-climbing question

Why searching is necessary

- Most AI problems have **no formula** that computes the answer. There is only a start state, a set of legal moves and a test for the goal, so the answer has to be **found** rather than calculated.
- Once a problem is written as a state space (Ch 2), **solving it is searching that space** for a path from initial state to goal.
- The space is far too large to enumerate: chess has around 10^{120} game paths, the 24-puzzle around 10^{25} states. **Brute force is not an option**, so the search must be directed.
- Search is the mechanism behind almost every other topic on this syllabus: game playing (§3.4), resolution refutation (Ch 4), rule chaining, planning, and the learning algorithms of Ch 6.
- **The one-line answer:** searching is how an agent converts knowledge of the rules into a sequence of actions, when it has no direct way to compute one.

The four evaluation criteria

- **Completeness** – is the algorithm guaranteed to find a solution when one exists?
- **Optimality** – when it finds one, is it guaranteed to be the *least-cost* solution?
- **Time complexity** – how many nodes are generated?
- **Space complexity** – how many nodes must be held in memory at once?
- **Say what the symbols mean** or the complexities score nothing:
 - b = **branching factor**, the maximum successors of any node;
 - d = **depth of the shallowest goal**;
 - m = **maximum depth** of the search tree, possibly infinite;
 - ℓ = the **depth limit**, for depth-limited search.
- Some papers add **admissibility** (does it find the optimal path when one exists) and **dominance** (does one heuristic expand fewer nodes than another). Mention both in a 4-mark answer.

3.2 Uninformed (blind) search

Explain BFS and DFS and compare their performance criteria 4 7 8 9

TOP 10/41 72 Ash 9, 69 Po 9, 70 Bh 4+4, 70 Ma 1+7, 73 Bh 4+5, 79 Bh 7, 79 Jth 7, 74 Bh 8, 71 Bh 8, 72 Ma 7 · the single most-asked question in the chapter · 71 Bh and 72 Ma additionally ask why these are called blind

What “uninformed” means

- The algorithm is given **nothing beyond the problem definition**: it can generate successors and test for the goal, and that is all.
- It has **no estimate of how close a state is to the goal**, so it cannot prefer one frontier node over another on merit. Hence **blind search**. That is the answer to “why are BFS and DFS called blind even though they search systematically”: systematic is not the same as informed.

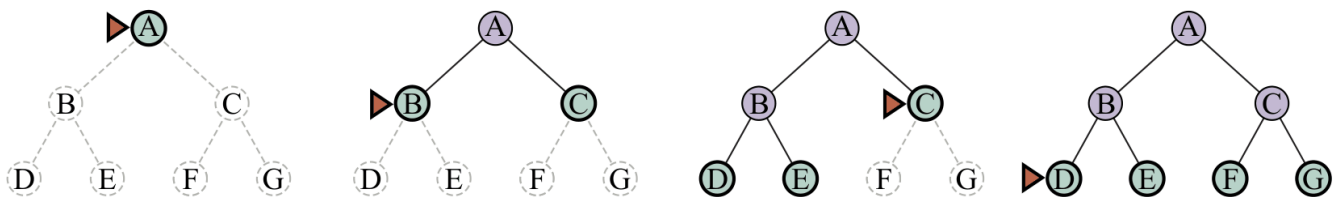
Breadth first search

- Expand **all nodes at depth k before any at depth $k + 1$** . **FIFO queue**.
- ✓ Complete, and ✓ optimal *when every step costs the same*.
- × Memory is the killer: it must hold the whole frontier, $O(b^d)$ nodes.

Depth first search

- Expand the **deepest** unexpanded node; go back only when stuck. **LIFO stack**, or plain recursion.
- ✓ Memory is linear, $O(bm)$: only the current path and its siblings.
- × Not complete (it can run down an infinite branch or loop), and × not optimal (it returns the first goal found, however deep).

The same tree, the same goal M , the two orders

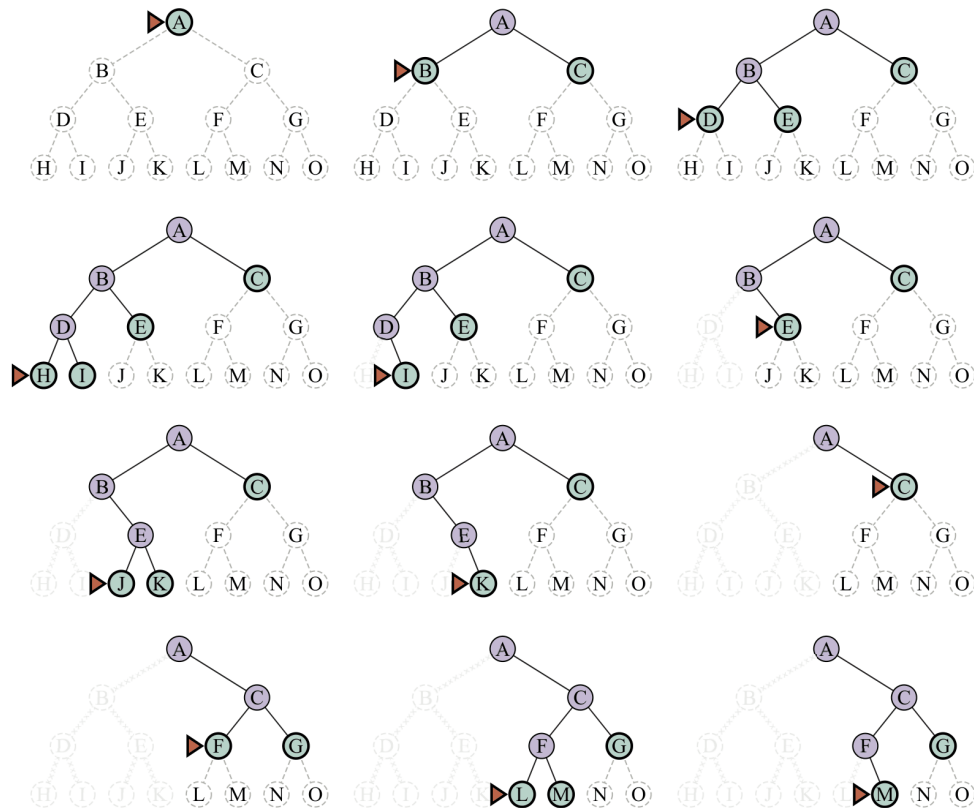


BFS: the frontier is a whole *level*. Four steps, and it has not gone below depth 2. Every node of a level is in memory at once, which is the $O(b^d)$ space.

The comparison table. Learn this.

	BFS	DFS	Why
Data structure	FIFO queue	LIFO stack	shallowest Vs deepest first
Complete	✓ yes	× no	DFS can descend forever
Optimal	✓ yes (unit step cost)	× no	BFS meets the shallowest goal first
Time	$O(b^d)$	$O(b^m)$	$m \geq d$, so DFS can be far worse
Space	$O(b^d)$	$O(bm)$	the real difference
Good when	the goal is shallow, memory is plentiful	the tree is deep, memory is tight, solutions are dense	

- **The sentence that earns the last mark**: BFS trades *memory* for a guarantee; DFS trades the *guarantee* for memory.
- With $b = 10$ and $d = 12$, BFS needs about 10^{12} nodes in memory and DFS about 120. That is the whole argument in one number.



Depth limited search (DLS) (71 Bh 4+4 , 79 Ash 4)

- Iterative deepening (IDS)
- verified/added

- Run DLS with $\ell = 0, 1, 2, \dots$ until a goal is found.
- **Gets BFS's guarantees at DFS's memory cost:** complete, optimal for unit steps, $O(b^d)$ time, $O(bd)$ space.
- The repeated work is negligible: re-expanding the top of the tree costs a constant factor, because the bottom level has more nodes than all the levels above it combined.
- **The preferred blind search** when the depth is unknown. Worth one line whenever a paper asks for a comparison of blind strategies.

Informed Vs uninformed search 3 4 8

HOT 4/41 74 Ma **8**, 81 Ba **4+4**, 81 Bh **3+6**, **72 Ash 8** · in all four papers it is the **first half of a hill-climbing question**, so answer it in a table and spend the time on the second half

	Uninformed (blind)	Informed (heuristic)
Extra knowledge	None beyond the problem definition	A heuristic $h(n)$ estimating the cost from n to the goal
Chooses by	Position in the queue or stack	The value of an evaluation function
Efficiency	Explores blindly, generates many useless nodes	Explores towards the goal, generates far fewer
Completeness	BFS, UCS, IDS yes; DFS no	A* yes; greedy and hill climbing no
Optimality	BFS and IDS for unit cost, UCS always	A* with an admissible h ; greedy never
Cost	Cheap per node	Needs a heuristic, which costs domain knowledge to design and time to evaluate
Examples	BFS, DFS, DLS, IDS, uniform cost	Best first, greedy, A*, hill climbing, simulated annealing

- **The line that makes the difference concrete:** on the same problem an uninformed search may generate millions of nodes where an informed one generates hundreds. The heuristic is not a different algorithm, it is **knowledge added to the same algorithm**.

3.3 Informed search: best first, greedy and A*

Explain A* search. How does it overcome the drawback of greedy best-first search? 2 4 6 8

HOT 7/41 73 Ma 8, 77 Ch 8, 78 Po 8, 73 Bh 8, 82 Ba 2+6, 78 Ba 6, 82 Bh 4+5 · short note on A* alone at **HOT 5/41** 72 Ma, 79 Jth, 81 Bh 4, 80 Ba, 76 Ba 3 · **always phrased as a comparison**, so never explain A* without saying what greedy gets wrong

The three evaluation functions

- $g(n)$ = the cost **already paid** to reach n from the start.
- $h(n)$ = the **estimated** cost from n to the goal, the heuristic.
- **Best first** search is the family: keep an **OPEN** list of generated but unexpanded nodes and a **CLOSED** list of expanded ones, and always expand the OPEN node with the best evaluation.
 - **Greedy best first:** $f(n) = h(n)$. Ignores what has already been spent.
 - **Uniform cost:** $f(n) = g(n)$. Ignores the goal. This is blind search.
 - **A*:** $f(n) = g(n) + h(n)$. **Uses both.**

The A* algorithm, as it should be written

1. Put the start node on OPEN with $g = 0$, $f = h(\text{start})$.
2. If OPEN is empty, fail.
3. Remove the node of **lowest** f from OPEN, call it n , and put it on CLOSED.
4. If n is the goal, stop; trace the parent pointers back for the path.
5. Expand n . For each successor m : compute $g(m) = g(n) + \text{cost}(n, m)$ and $f(m) = g(m) + h(m)$; record n as its parent; and if m is already on OPEN or CLOSED with a **larger** g , **replace the old entry and reopen it**.
6. Re-order OPEN by f and go to 2.

Admissibility and optimality of A* 4+5

PIN 1/41 82 Bh 4+5 · asked as “define admissibility and optimality; under what conditions is A* guaranteed to be both”, so the two **conditions** are the answer

The two properties of a heuristic

- **Admissible:** $h(n) \leq h^*(n)$ for every n , where h^* is the true cost to the goal. The heuristic **never overestimates**; it is optimistic.
 - Guarantees A* is optimal in **tree search**.
- **Consistent (monotone):** $h(n) \leq \text{cost}(n, m) + h(m)$ for every edge, and $h(\text{goal}) = 0$. A triangle inequality on the heuristic.
 - Guarantees A* is optimal in **graph search**, i.e. with a CLOSED list that is never revisited. Consistency implies admissibility; the converse is false.
- **Conditions for A* to be admissible and optimal:** h admissible, every step cost > 0 , and the branching factor finite. Add *consistent* if a closed list is used.

Greedy's drawback, and A*'s fix

- Greedy asks only “which node looks nearest to the goal?”. It therefore **walks into whatever the heuristic flatters**, and a long detour that starts with a promising-looking step will be taken in full.
- $\times$ Greedy is **not optimal** and **not complete**: it can loop between two nodes that each look good from the other.
- A* adds $g(n)$, so **a node is judged on the whole journey, not the remainder**. A cheap-looking node that cost a fortune to reach is ranked accordingly.
- **The worked case in the companion:** on the 80 Ch graph, greedy takes $S \rightarrow C \rightarrow D \rightarrow F \rightarrow G$ at cost **25**, which is the *worst* of the four routes, because $h(C) = 5$ looks closest. A* takes $S \rightarrow B \rightarrow E \rightarrow F \rightarrow G$ at cost **18**, the optimum.

Comparison in one line each

- **Greedy:** fast, small memory, no guarantees.
- **A*:** optimal and complete with an admissible h , but stores every generated node, so **memory is its limit**.

Why this is not a pedantic point

- Both graphs printed in these papers carry heuristics that are **admissible but not consistent**. In 81 Ch, $h(B) = 13$ while $\text{cost}(B, E) + h(E) = 6 + 4 = 10$.
- The consequence is visible in the trace: A* first reaches E at $g = 12$ through A , and later finds E at $g = 11$ through B . **An implementation that refuses to reopen a closed node returns 19 instead of the optimal 18.**
- So step 5 of the algorithm above, the *replace and reopen* clause, is not decoration. Write it, and the trace comes out right.
- **Dominance:** if $h_2(n) \geq h_1(n)$ for all n and both are admissible, h_2 *dominates* h_1 and A* with h_2 expands no more nodes. A better heuristic is one that is larger while staying under h^* .

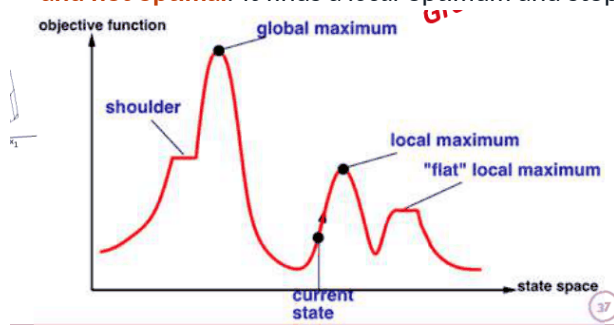
3.4 Hill climbing and simulated annealing

Discuss hill climbing, the problems associated with it, and their solutions 3 4 6 9

HOT 6/41 69 Bh 9, 75 Ba 6+2, 80 Ba 3+5, 75 Bh 4, 77 Ch 4, 79 Bh 4 · **three problems, three cures**, and the marks are split evenly between them. Four of these six papers then ask how simulated annealing fixes it

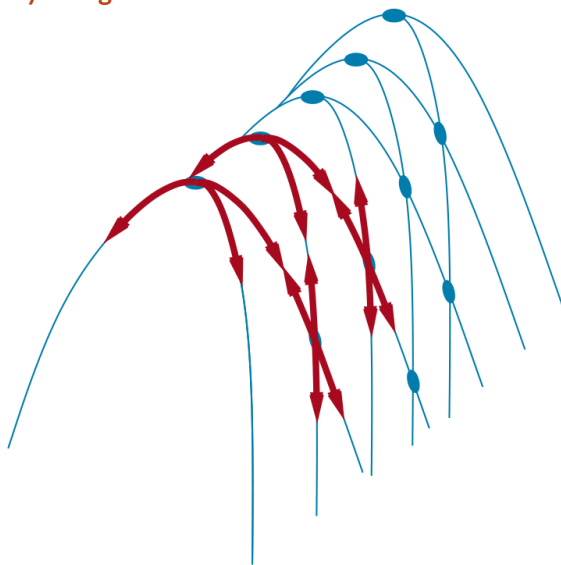
The algorithm

- A **local search**: keep only the current state, and move to a neighbour that improves the objective function. No frontier, no backtracking, no memory of where it has been.
- **1.** Evaluate the initial state; if it is the goal, stop. **2.** Generate a neighbour. **3.** If the neighbour is better, make it the current state. **4.** Repeat until no neighbour is better.
- **Variants**: *simple* hill climbing takes the first better neighbour; *steepest ascent* evaluates all neighbours and takes the best; *stochastic* picks randomly among the better ones.
- ✓ Almost no memory, and very fast. × **Not complete and not optimal**: it finds a *local* optimum and stops.



The whole answer in one picture: climbing from the current state reaches the local maximum, not the global one.

Why a ridge is the hard one

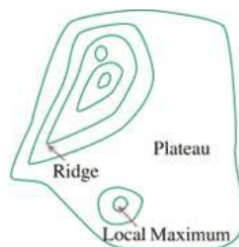


The three problems, and their cures

Problem	What happens	Cure
Local maximum	A peak higher than all its neighbours but lower than the global maximum. Every move looks worse, so the search stops on a suboptimal answer.	Backtrack to an earlier node and try a different direction. Or restart from a new random state (random-restart hill climbing).
Plateau	A flat region where all neighbours score the same, so the search has no gradient to follow and wanders at random.	Make a big jump in some direction, to reach a new region of the space. A single step is not enough to get off it.
Ridge	A sequence of local maxima along a slope where the ascent runs <i>across</i> the ridge rather than along it, so every single-step move goes down.	Apply two or more rules at once before testing, i.e. move in several directions at a time. Bidirectional search also helps.

- **Name all three and pair each with its own cure.** A generic “use random restarts” for all three loses half the marks.

- The dots are **states**; they sit on a ridge rising left to right. The dark arrows are the moves available from each state.
- **Every one of those moves goes down.** The ascent runs *across* the ridge, not along it, so no single step improves anything and hill climbing halts on a slope that is not a maximum at all.
- That is exactly why the cure is **apply two or more rules at once**: one step in any single direction cannot go up, but a combination can.
- **A ridge needs two dimensions to exist**, which is why the one-dimensional landscape on the previous page cannot show one. If a paper asks for “the problems of hill climbing with diagrams”, **this is the one people leave out**.



The same landscape as a contour map, which shows all three problems at once: the **plateau** is the wide gap between contours, with no gradient anywhere on it; the **ridge** is the narrow crest on the left; the **local maximum** is the small closed loop at the foot.

Simulated annealing 4 5 6

HOT 4/41 75 Ba 2, 80 Ba 5, 81 Ba 4, 81 Bh 6 · **not a syllabus line**, but asked four times and always as “how does it overcome the demerits of hill climbing”

The idea

- Named after **annealing** in metallurgy: heat a metal, then cool it slowly so its atoms settle into a low-energy crystal instead of freezing into a flawed one.
- Hill climbing accepts *only* improving moves, which is exactly why it gets stuck. **Simulated annealing sometimes accepts a worse move**, and does so less and less as the search proceeds.
- A move is picked at random:
 - if it **improves** the state, accept it always;
 - if it **worsens** the state by ΔE , accept it with probability $P = e^{-\Delta E/T}$.
- T is the **temperature**, lowered on a fixed **annealing schedule**.

Why that fixes hill climbing

- An accepted downhill move is exactly what **escapes a local maximum**: the search can go down one side of the small peak and up the big one.
- It also **crosses a plateau**, since a sideways or slightly worse move costs almost nothing in probability.
- **The role of T** , which is where the marks are:
 - **High $T \rightarrow P$ near 1**, most bad moves accepted, the search explores wildly. Nearly a random walk.
 - **Low $T \rightarrow P$ near 0**, bad moves refused, the search becomes **ordinary hill climbing**.
 - Starting hot and cooling slowly explores first and refines later.
- Note the shape of $e^{-\Delta E/T}$: a *slightly* worse move is much likelier to be accepted than a much worse one, at any temperature.
- **The theoretical guarantee worth quoting**: if T is lowered slowly enough, simulated annealing finds the global optimum with probability approaching 1.

3.5 Adversarial search: minimax and alpha-beta

Explain the min-max algorithm. What are its drawbacks, and how does alpha-beta pruning solve them? 2 3 4 6

HOT 6/41 71 Ma 2+4+3, 77 Ch 2+6, 82 Ka 6, 81 Ash 6, 76 Bh 4, 80 Bh 2+6 · short notes on the same at **HOT 6/41** 78 Ba, 74 Bh, 74 Ma, 72 Ash, 78 Po 4, 82 Ba 3 · **12 papers between them**, which makes this the second-biggest topic in the chapter

Minimax

- For a **two-player, zero-sum, perfect-information** game. One player (**MAX**) maximises the score, the other (**MIN**) minimises it.
- Build the game tree to the terminal states, score each with the utility function, then **propagate values upward**:
 - at a **MAX** node, take the **largest** child value;
 - at a **MIN** node, take the **smallest**.
- The root’s value is the best MAX can force **against an opponent playing perfectly**, and the move to play is the child that achieves it.
- Implemented as a **depth-first** traversal, so space is $O(bm)$; time is $O(b^m)$.

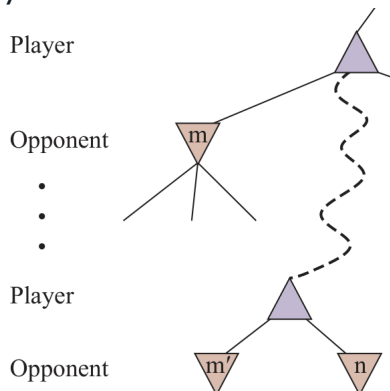
The drawbacks (the part the papers ask for)

- **Exponential time**, $O(b^m)$. Chess has $b \approx 35$ and $m \approx 100$, so the complete tree can never be built. **This is the drawback**; everything else is a consequence.
- It **examines every leaf**, including branches whose outcome cannot possibly change the root value.
- It assumes a **perfect opponent**, so it can play too cautiously against a weak one.
- In practice the tree must be **cut off at a fixed depth** and the cut-off nodes scored by an **evaluation function**, which makes the answer approximate.

Alpha-beta pruning

- **Same answer, fewer leaves.** It returns exactly the minimax value; it only skips work that cannot affect it.
- Carry two bounds down the tree:
 - α = the best value **MAX** can already guarantee. It only rises.
 - β = the best value **MIN** can already guarantee. It only falls.
- **The two cut rules:**
 - at a **MIN** node, if its value $\leq \alpha$, **stop** – MAX will never choose this branch (*alpha cut-off*);
 - at a **MAX** node, if its value $\geq \beta$, **stop** – MIN will never allow it (*beta cut-off*).

Why a cut is safe



The whole justification in one picture. Player is choosing at the top and can already reach m . Somewhere far below, n is a node Player would have to pass through to get anywhere in that branch, and Opponent can hold Player to m' there. · **If m or m' is better than n for Player, then Player will never be allowed to reach n in real play** – so whatever value n turns out to have cannot change the root, and the whole subtree under it can be skipped unexamined. · That is the sentence to write when a paper asks *why* pruning does not change the answer: a pruned branch is one a rational player above it would never let the game enter.

How to run alpha-beta in an exam, without losing the thread

- **Go strictly left to right, depth first.** The examiner's expected answer depends on that order.
- Write (α, β) beside every internal node as you enter it, starting $(-\infty, +\infty)$ at the root.
- A child **inherits** its parent's current α and β .
- After each child returns, update the node's own value, then update α (at a MAX node) or β (at a MIN node).
- Test the cut rule **immediately after each child**, not at the end. Cross out the skipped leaves so the pruning is visible.
- **State how many leaves were pruned** at the end. It is usually worth a mark and it proves the trace was actually run.

What pruning buys, and what it does not

- **Effect:** with a *perfect* move ordering the time complexity falls from $O(b^m)$ to $O(b^{m/2})$, i.e. **the same time searches twice as deep**, or an effective branching factor of $\sqrt{b}$. With random ordering it is about $O(b^{3m/4})$.
- × It does **not** change the value returned.
- × It does **not** help in the worst case: with the best move examined last everywhere, nothing is pruned and the cost is exactly minimax's.
- So it depends entirely on **move ordering**, which is why real game programs invest in ordering heuristics before they invest in searching deeper.

Worked in the companion

- 79 Ch: a 12-leaf, 5-level tree. Minimax value 3; alpha-beta prunes **2 of the 12 leaves**.
- 76 Ba: a 9-leaf, 2-level tree. Minimax value 3; alpha-beta prunes **4 of the 9**, because the first branch returns 3 and sets a high α straight away.
- **The contrast between the two is the lesson:** the second tree prunes twice as large a fraction, purely because its best branch happened to come first.

Other adversarial ideas worth a line

- **Horizon effect:** a fixed-depth cut-off can hide a disaster just beyond the last ply, so the agent plays a move that merely postpones it.
- **Quiescence search:** keep searching past the cut-off while the position is still volatile.
- **Expectiminimax:** for games with chance, add a *chance* node whose value is the expected value over its children.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network

Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

Three course codes, three faces. Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.